

throwing an exception by own using throw keyword :

```
void main()
{
    IO.println("Hello Learners!!!");
    try
    {
        //IO.println(10/0);
        throw new ArithmeticException("I am dividing a number by zero");
    }
    catch(Exception e1)
    {
        IO.println(e1);
    }
    IO.println("Thank you");
}
```

From the above program it is clear that try block implicitly creating the appropriate exception object with the help of JVM and throwing the exception object to the nearest catch block.

We can also create the exception object manually and can throw the exception object to the catch block as shown in the above program

After throw keyword we can't write any kind of statement, It will become un-reachable code.

```
class Foo
{
}

public class ExceptionDemo
{
    public static void main(String[] args)
    {
        try
        {
            throw new Foo();
        }
        catch (Exception e)
        {
            System.out.println(e);
        }
    }
}
```

Note : We will get compilation error because Foo class does not belong to exception hierarchy so Foo class not a Throwable object.

Note : The main purpose of exception handling to provide user-friendly message so, client can enjoy the services of software/websites.

Exception handling = Normal Termination + User-friendly message on wrong input given by the client.

```
package com.ravi.basic;

public class CustomerDemo {

    public static void main(String[] args)
    {
        IO.println("Hello Client, Welcome to my Application :");

        try
        {
            int x = Integer.parseInt(IO.readLine("Enter the value of x : "));
            int y = Integer.parseInt(IO.readLine("Enter the value of y : "));
            int z = x / y;
            IO.println("Result is : "+z);
        }
        catch(Exception e)
        {
            System.err.println("Donot put 0 (zero) here");
        }

        IO.println("Thank you for visiting my application ");
    }
}
```

Throwable class Methods to print Exception :

a) public String toString() : FQN: error message [Convert the Exception object into String]
b) public String getMessage(): Only error message
c) public void printStackTrace() : Will provide the complete exception details i) FQN: error message ii) class name iii) Method Name iv) Line numbers

Throwable class has provided the following three non static methods :

- 1) public String getMessage() :- It will provide only error message.
- 2) public void printStackTrace() :- It will provide the complete details regarding exception like exception class name, exception error message, exception class location, exception method name and exception line number.
- 3) public String toString() :- It will convert the exception into String representation to provide Fully Qualified Name: error message

```
//Program :
-----
package com.ravi.basic;

public class ThrowableMethods {

    public static void main(String[] args)
    {
        try
        {
            int []arr = {12, 90, 56, 78, 90, 17};

            IO.println(arr[6]);
        }
        catch(Exception e)
        {
            IO.println(e.toString()); //FQN: errorMessage
            IO.println(".....");
            IO.println(e.getMessage()); //Only error message
            IO.println(".....");
            e.printStackTrace();
        }
    }
}
```

Working with Specific Exception :

While working with exception, in the corresponding catch block we can take Exception (super class) which can handle any type of Exception.

On the other hand we can also take specific type of exception (ArithmeticException, InputMismatchException and so on) which will handle only one type i.e specific type of exception.

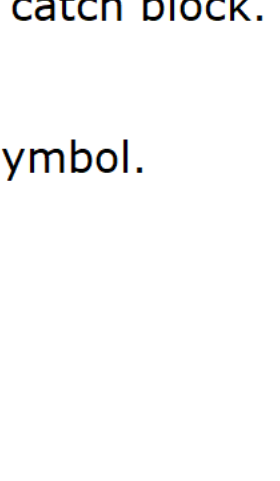
```
public class SpecificDemo
{
    public static void main(String [] args)
    {
        try
        {
            String str = "Java";
            int num = Integer.parseInt(str);
            IO.println(num);
        }
        catch(NumberFormatException e) //Specific Exception
        {
            IO.println("Catch Block");
            IO.println(e);
            IO.println(e.getMessage());
        }
    }
}
```

try with multiple catch blocks :

```
try
{
    10/0; 5 //AE

    IO.println(arr[ 5 ]); //AIOOBE
}
catch(ArrayIndexOutOfBoundsException e)
{
    IO.println("Array index is out of bounds");
}
catch(ArithmeticException e)
{
    IO.println("Divide by zero prob");
}
catch(Exception e)
{
    IO.println("Some General Problem "+e);
}
```

```
try
{
    10/2;
    IO.println(arr[10]);
}
catch(Exception e)
{
    IO.println("Divide by zero Prob");
}
```



According to industry standard we should write try with multiple catch blocks so we can provide proper information for each and every exception to the end user (client).

While working with multiple catch block always the super class catch block must be last catch block. [CE : Unreachable code]

From java 1.7v this multiple exceptions we can write in a single catch block by using | symbol.

```
Example :
try
{
}
catch(ArithmeticException | NullPointerException e)
{
}
```

If try block is having more than one exception then always try block will entertain only first exception because control will transfer to the nearest catch block.